# High Performance Computing

Francesco Lovergine

Mar 2000

# Parallelismo e calcolo

Un computer *parallelo* è un sistema di processori che è in grado di lavorare in cooperazione per risolvere un problema computazionale.

Questa definizione include supercomputer che hanno centinaia o anche migliaia di processori, reti di computer, computer multiprocessore con pochi processori o anche sistemi *embedded* dedicati alla esecuzione di specifici task.

Il parallelismo è stato sempre visto come un'area di indagine "esotica" del calcolo, interessante, ma complessa è di scarsa rilevanza per il programmatore medio.

## IN REALTÀ

*Esistono problemi computazionali semplicemente non risolubili se non facendo ricorso al supercalcolo, sia in area applicativa (es. metereologia) che nel campo della ricerca pura*.

## MA

*L'implementazione di pradigmi paralleli è generalmente impegnativa in termini di disegno, test, debugging in quanto assolutamente* non trasparente *per il developer.*



# Un modello di sistema parallelo

La macchina sequenziale di Von Neumann è sottesa alla stragrande maggioranza degli algoritmi che risolvono un problema di calcolo. Non necessariamente tale macchina è il modello computazionale più adeguato e non necessariamente l'algoritmo prescelto è il più adatto a risolvere il problema.

Si possono definire diversi modelli di macchina parallela per implementare un medesimo algoritmo, allo stesso modo. Ogni modello deve consentire di massimizzare

| concorrenza – scalabilità – località |
|---|

e a tale fine si può introdurre la

*tassonomia di Flynn*

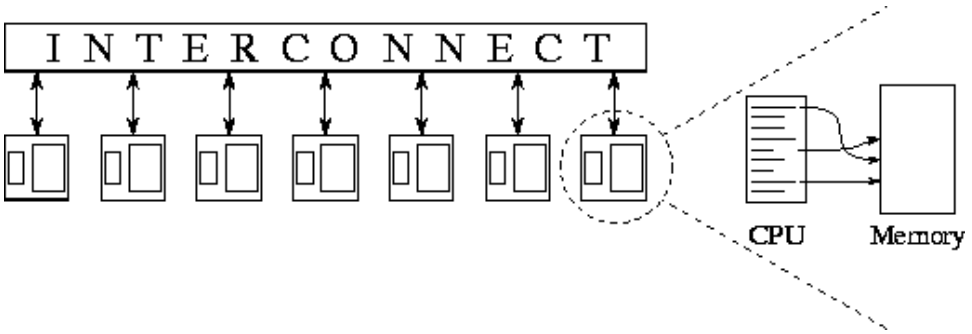
**Multicomputer** È costituito da un certo numero di macchine di Von Neumann o nodi, collegati fra loro da una rete di interconnessione. Ogni computer esegue il proprio programma e accede alla memoria **locale** o può inviare e ricevere **messaggi** attraverso la rete. I messaggi sono utilizzati per comunicare con gli altri computer oppure equivalentemente per leggere o scrivere nelle memorie remote. In una situazione ideale il costo di invio/ricezione



di un messaggio dipende solo dalla dimensione del messaggio stesso. Le primitive essenziali di comunicazione sono

- send
- receive
- read
- write

In generale il costo di send/receive è maggiore del costo di read/write. Per questo motivo è desiderabile massimizzare la località. L'importanza della località dipende dal rapporto tra costo di accesso locale e remoto.




**Distributed memory MIMD** È il modello più simile al multicomputer. Anche in questo caso molti processori sono connessi fra loro da una rete di interconnessione. MIMD (Multiple Instruction Multiple Data) sta a indicare che ogni processore può eseguire un flusso di istruzioni sui propri dati locali; la memoria distribuita indica il fatto che la memoria totale è divisa tra i processori e non concentrata centralmente e accessibile contemporanea-

mente da tutti i processori. La differenza tra dm-MIMD e il multicomputer sta nel fatto che il costo di invio di un messaggio dipende dalla localizzazione del processore nel network e dal traffico nel network. La rete di fatto potrebbe non essere full-meshed.

**Shared Memory MIMD** Detto anche *multiprocessore*. In questo caso tutti i processori condividono una memoria comune, tipicamente attraverso un bus o una gerarchia di bus. Nella macchina ideale (PRAM – Parallel Random Access Machine) ogni processore può accedere alla memoria nella stessa quantità di tempo. In pratica vengono generalmente introdotte delle cache memory locali per minimizzare il più possibile i tempi di accesso alla memoria condivisa e quindi torna ad essere importante la località. La differenza tra multiprocessori e multicomputer diventa relativa a questo punto. Programmi pensati per essere efficienti su multicomputer lo sono anche su multiprocessori (cfr implementazione di MPI) in quanto la shared memory consente una piuttosto efficiente del message passing. L'elaborazione è in generale ASINCRONA–NON BLOCCANTE.

**SIMD** È un modello molto specializzato di computer parallelo. Nella architettura SIMD (Single Instruction Multiple Data) ogni processore esegue lo stesso flusso di istruzioni su dati differenti. Questo approccio può ridurre la



complessità dell'hardware e del software di base ma è specializzato per problemi caratterizzati da una elevata "regolarità".

Algoritmi per multicomputer non possono in generale essere eseguiti efficientemente su macchine SIMD.

APE 100 ($\Rightarrow$ Quadrics) è un esempio di tale architettura (in realià SISAMD). L'elaborazione è sempre SINCRONA–BLOCCANTE.

Le macchine SIMD possono essere il modello più adeguato per specifici algoritmi numerici, ma la rispondenza deve essere accuratamente valutata.

Le caratteristiche intrinseche del modello SIMD impongono che i processori (o PE – processing element) di tali macchine debbano essere *elementari*, ovvero dotati di un set di istruzioni estremamente ridotto ed in numero estremamente elevato.

# PERCHÉ?



# Soluzioni architetturali

Ogni modello della tassonomia, nel momento in cui viene implementato presenta problemi che possono essere anche notevoli, al di là delle limitazioni teoriche proprie dello specifico modello.

Un vantaggio di una macchina MIMD è la apparente facilità di programmazione in quanto l'accesso a risorse condivise viene regolato usando tecniche sviluppate per i computer multitasking (semafori) quando necessario. Tuttavia le archiettura SM non scalano generalmente bene. Al crescere del numero di processori aumenta la frequenza di conflitti in accesso al bus. Un modo di ridurre tale problema è quello di dividere la memoria in modo trasparente in un serie di moduli ognuno connesso a un processore mediante un network ad alte prestazioni.

Il problema si sposta a quel punto sul network: all'aumentare del numero di processori non è praticabile una rete totalmente connessa e ciascun processore è collegato solo a un insieme ridotto di processori. Ogni processore per raggiungerne un altro può dover "routare" attraverso diversi altri. Una topologia di connessione spesso usata in tale contesto è l'*ipercubo* che ha il vantaggio di avere percorsi ridondanti di pari lunghezza e poche interconnessioni paritarie.



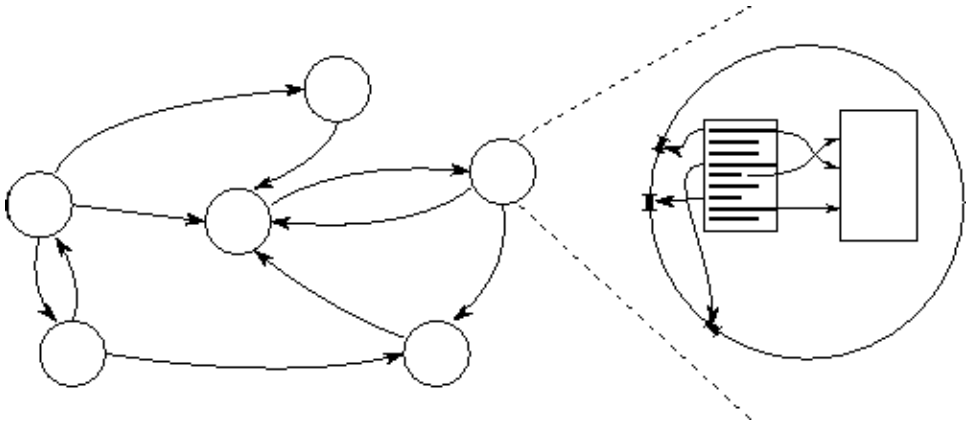
# Un modello di programmazione parallela

Come per la macchina parallela possono essere definiti diversi modelli astratti di programmazione parallela, che consentono l'astrazione e favoriscono quindi la riduzione della complessità. La quarta caratteristica da rispettare per il software parallelo è la

modularità

necessaria per il controllo della maggiore complessità.

Il modello di programmazione più naturale è il paradigma *task/channel*, ovvero un insieme dinamico di processi in esecuzione contemporanea, che attivano fra loro dinamicamente uno o più canali di comunicazione.



Le caratteristiche di tale paradigma sono:

1. Il numero di task **varia** durante l'esecuzione.
2. Un task comprende un programma sequenziale e una memoria locale (modello Von Neumann) e un insieme di porte di in/out che definiscono la sua interfaccia con gli altri task.
3. Oltre a leggere e scrivere in memoria locale, un task può inviare e ricevere un messaggio, crere nuovi task e terminare.
4. Una operazione di invio è asincrona, una operazione di invio è generalmente sincrona.
5. Le coppie di porte I/O possono essere connesse da *code* chiamate *canali*. I canali possono essere creati e cancellati e variano quindi di numero dinamicamente in corso di esecuzione. La presenza di un canale implica la *data dependency* fra due task.
6. I task possono essere mappati ai processori in vari modi e tale mapping non influenza la semantica del programma. In particolare più task possono essere mappati su un singolo processore.

Il paradigma task/channel rispetta quattro linee guida fondamentali



**Performance**

**Mapping independece**

**Modularity**

**Determinismo**

**Performance** L'astrazione del computer sequenziale è efficiente in quanto mappabile direttamente sul modello Von Neumann. Allo stesso modo il paradigma *task/channel* è mappabile efficacemente su una qualsiasi architettura parallela. Nel caso di multiprocessore, il canale viene implementato con l'IPC, altre tecniche vengono impiegate per differenti architetture.

**Mapping independece** Gli algoritmi possono essere implementati indipendentemente dal numero di processori reali utilizzati. Questo consente di ottenere programmi scalabili senza modificare il codice.

**Modularity** Nel disegno modulare di programmi, le varie componenti vengono progettate come *black-box*, definendo le interfacce dei vari moduli. La implementazioni dei singoli moduli possono quindi essere facilmente modificate senza side-effects sugli altri moduli. Questo approccio



riduce la complessità e aumenta la riusabilità del codice. Il task risulta un building-block naturale per la programmazione modulare. Analogie esistono del resto tra il ben noto modello object-oriented e il mdoello task/channel: il task come l'oggetto, incapsula dati e metodi e definisce le sole interfacce, ma al contrario dell'oggetto non supporta l'ereditarietà.

**Determinismo** È altamente desiderabile un modello programmativo che consenta facilmente la costruzione di programmi deterministici. Il modello task/channel consente ad esempio di garantire il determinismo in modo relativamente semplice, è sufficiente garantire che ciascun canale abbia un unico sender e un unico receiver e che un task si blocchi in ricezione sino a quando l'operazione di ricezione non sia stata completata.



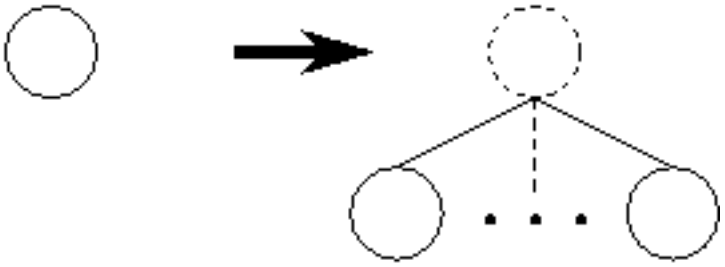
Send a message:



Receive a message:



Create tasks:



Terminate:



# Altri modelli di programmazione parallela

Il modello task/channel è sicuramente il più adatto a descrivere un algoritmo, ma sicuramente non è l'unico considerabile a tale scopo.

**Message passing** È in assoluto il modello di programmazione parallela più utilizzato. Come nel modello task/channel, il message passing si fonda sulla creazione di task multipli, ognuno dei quali incapsula i propri dati locali e comunica con i gli altri task inviando ad essi dei messaggi e ricevendone. Da questo punto di vista il modello differisce solo nel fatto che i messaggi non vengono inviati sul canale ma ad uno specifico task. La definizione di canale è una utile astrazione anche nel disegno di programmi message-passing.

Normalmente il modello message-passing più comune è il SPMD (*Single Progam Multiple Data* ) che prevede la creazione *una tantum* di tutti i task richiesti allo startup del programma è nessuna creazione dinamica di task seguente..

**Data parallelism** Il paradigma *data parallel* si basa sulla applicazione delle stesse operazioni concorrentemente su elementi diversi della stessa struttura di dati. Pertanto è molto adatta ad una implementazione su una architettura SIMD. Poiché il processing su ciascun elemento



deve essere considerato come un task indipendente, la granularità di un programma data parallel è piccola e il principio di locaità non appare. Per questo motivo. la distribuzione dei dai sui processori deve necessariamente essere fissata esplicitamente dal programmatore.

I compilatori possono tradurre un programma data-parallel in una formulazion SPMD, generando il codice di comunicazione automaticamente nel caso di architettura multicomputer/multiprocessore.

**Shared memory** I task condividono l'area dati e quindi viene meno la necessità di passare dati da un task ad un altro. È però neccessario prevedere meccanismi di gestione dell'accesso concorrente (locks e semafori). Il modello è apparentemente semplificato rispetto ai precedenti tuttavia la gestione della località è più complessa e può diventare difficle scrivere programmi deterministici.



# Esempi di algoritmi paralleli

I primi due esempi hanno una struttura SPMD, il terzo crea task dinamici e l'ultimo usa un numero fissato di task che svolgono funzioni differenti.

## [Esempio A]

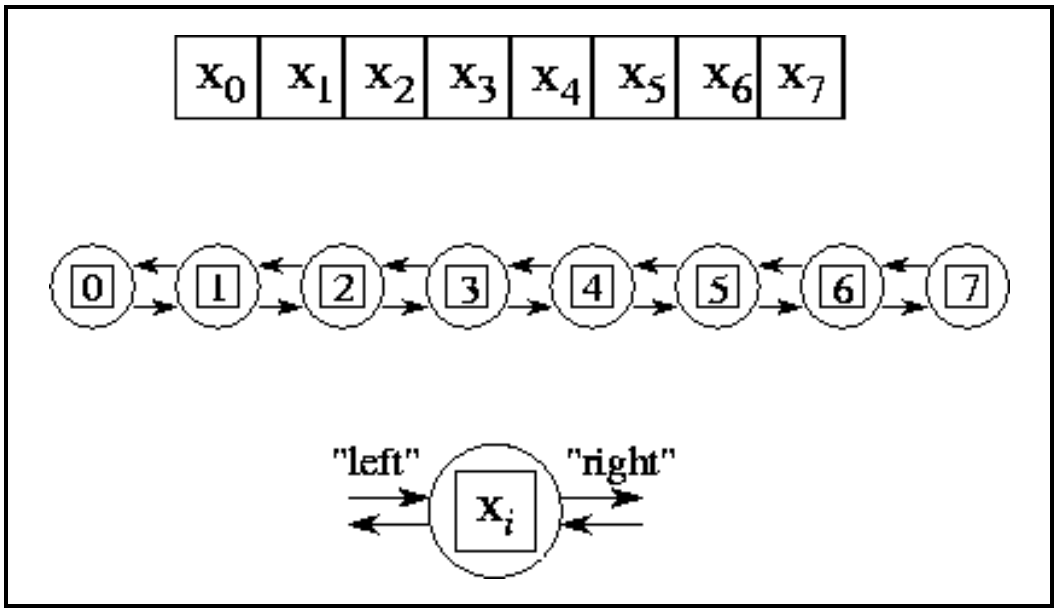



Figura 1: *Problema alle differenze finite unidimensionale. Dall'alto in basso, il vettore unidimensionale $X$ con $N = 8$; la struttura a task, ognuno dei quali incapsula un singolo valore e connette via channel un precedente e un seguente; la struttura di un singolo task, con le sue porte di I/O*

Si calcoli una successione cosí definita

$$X_i^{(t+1)} = \frac{X_{i-1}^{(t)} + 2X_i^{(t)} + X_{i+1}^{(t)}}{4}$$

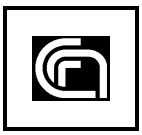
Definendo un task per ciascun valore $X_i$ la sincronizzazione deve assicurare che nessun valore sia aggiornato prima del completamento dello step precendente dei vicini prossimi. L'esecuzione è deterministica.

COSA ACCADE SE SI INTRODUCE UNA DIPENDENZA ON STEP?

$$X_i^{(t+1)} = \frac{X_{i-1}^{(t+\mathbf{1})} + 2X_i^{(t)} + X_{i+1}^{(t)}}{4}$$

Si noti che questo tipo di problema è MOLTO comune in analisi numerica.

## [Esempio B]



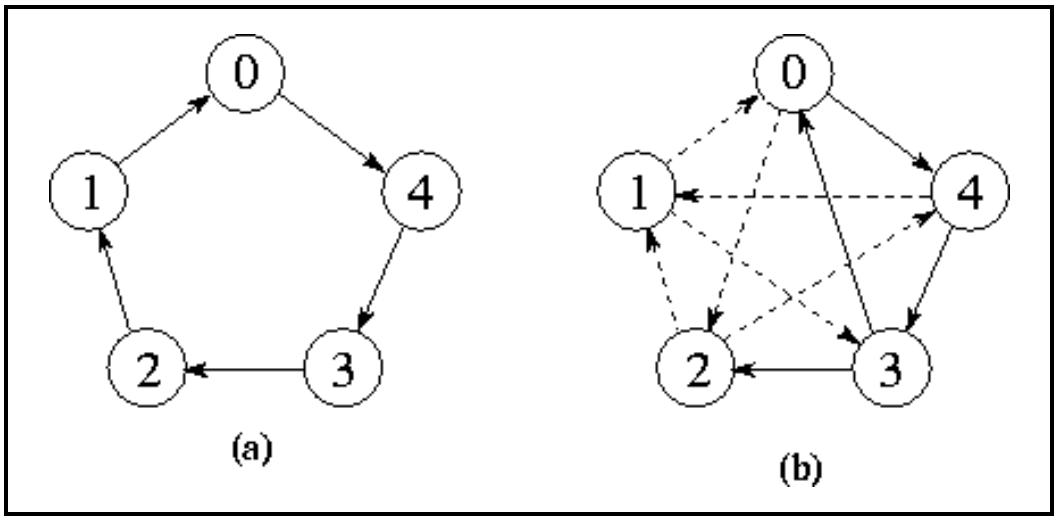



Figura 2: *Problema di interazione a coppia per $N = 5$. In (a) anello unidirezionale nel semplice algoritmo non simmetrico; in (b) l'anello unidirezionale con aggiunta di canali addizionali usati per ritornare i valori accumulati nell'algoritmo simmetrico.*

Molti problemi richiedono il calcolo di interazioni a coppie $I(X_i, X_j)$ con $i \neq j$. Le interazioni possono essere simmetriche ovvero $I(X_i, X_j) = \pm I(X_j, X_i)$ e quindi vanno calcolate solo $N(N-1)/2$ interazioni. Un algoritmo parallelo per questo genere di problema può prevedere la creazione di $N$ task ognuno dei quali calcola l'interazione $I(X_i, X_j)$. Invece di $N(N-1)$ canali si possono creare $N$ canali.

COME?

## [Esempio C]

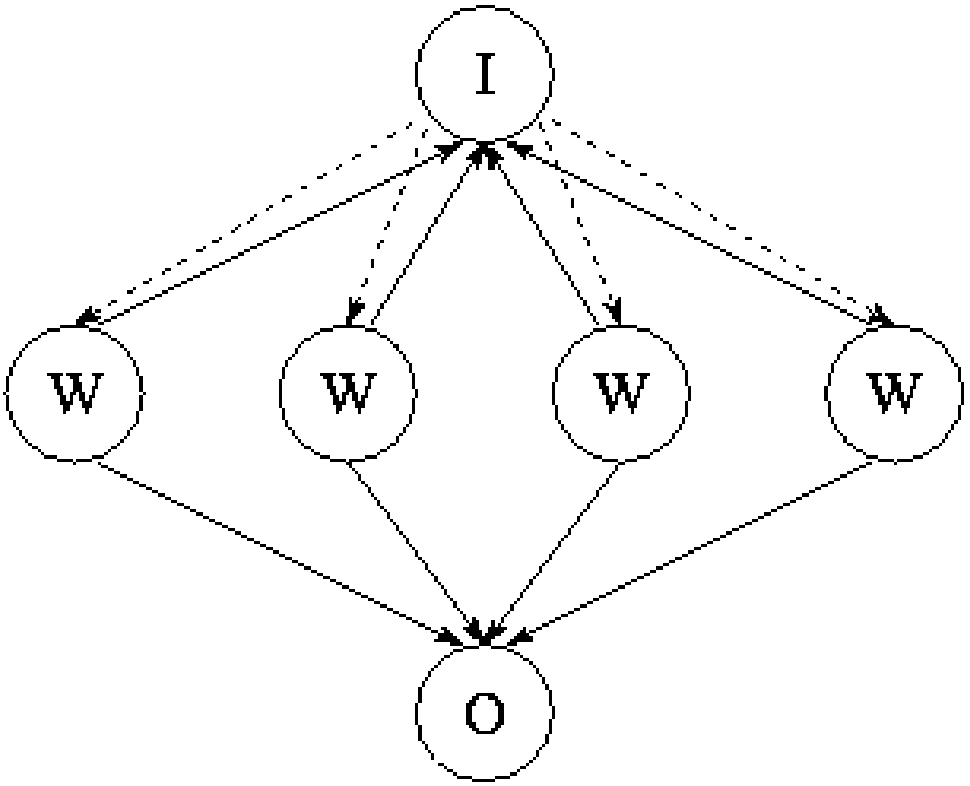
```
procedure search (A)
begin
if solution(A) then
    score=eval(A)
    report the solution and score
else
    foreach child A(i) of A
       search A(i)
    endfor
endif
end
```

Il problema di ricerca di può essere implementato con una gerarchia di processi che vengono creati dinamicamente man mano che vengono esplorati i nodi dell'albero che costituisce la struttura di dati.

È DETERMINISTICO?



# [Esempio D]




Lo studio di parametri è un esempio di programma *embarassingly data parallel*. Un tale tipo di problema può essere risolto da un programma che attiva un certo numero di task totalmente indipendenti su dati disaccoppiati. Supponiamo che lo stesso calcolo voglia essere effettuato per differenti parametri di input distinti. Se il tempo di esecuzione è costante e ciascun processore identicamente performante si può partizionare i dati in domini delle stesse dimensioni e allocare ciascuno ad un processore. È un esempio di pro-

gramma che usa strutture di comunicazione *many-to-one* nel quale l'ordine di presentazione dei dati (in input e output) non è determinato in anticipo.

PERCHÉ?



# Disegno di algoritmi paralleli.

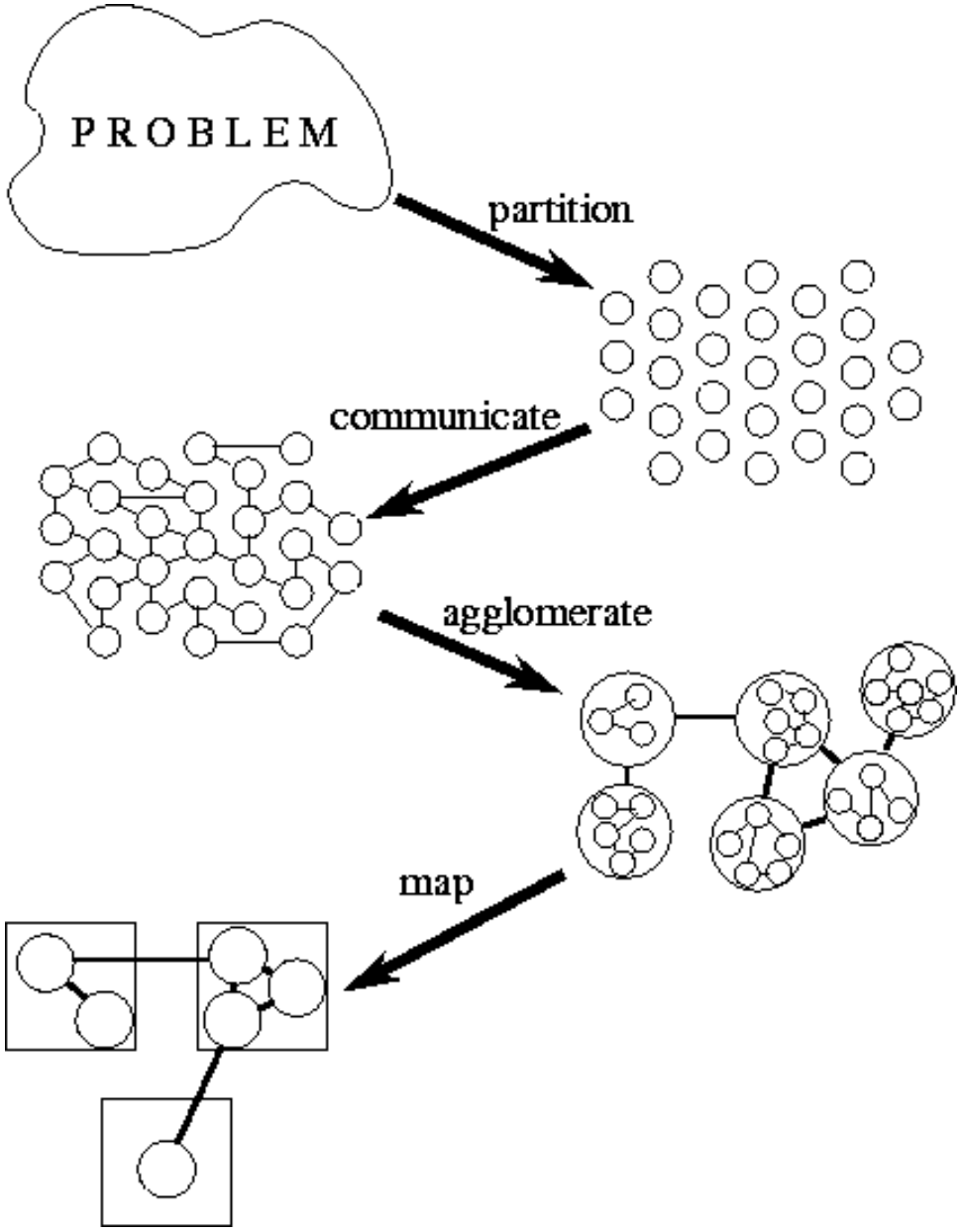
La maggior parte dei problemi computazionali sono suscettibili di più di una soluzione parallela. La soluzione migliore può anche essere diversa dalla migliore soluzione sequenziale. Una corretta metodologia di disegno dovrebbe il più possibile essere avulsa dalla particolare architettura di macchina parallela, per lo meno fino allo stadio finale, che dovrebbe curare le performance complessive e quindi adattare il programma alle caratteristiche della piattaforma.

La metodologia PCAM proposta si compone di quttro fasi, le prime due focalizzate su concorrenza e scalabiità e le ultime due su località e performace in genere.

Il risultato del PCAM può essere un programma che crea e distrugge task dinamicamente, utiizzando il bilanciamento di carico per mappare i task sui processori disponibili. Alternativamente potrebbe essere un programma SPMD che crea esattamente un task per processore, ma in ogni caso i due programmi arrivano a differenziarsi solo nell'ultima fase.

La procedura di disegno è sequenziale (!) tuttavia può presentare come naturale una reiterazione di una o più fasi.






1. *Partitioning* – L'elaborazione da svologere e i dati su cui tale elaborazione deve essere svolta, vengono suddivisi in chunck o task più piccoli. Aspetti "pratici" come il numero di processori disponibili, devono essere ignorati,

l'attenzione è focalizzata sulle possibilità di esecuzione concorrente.

2. *Communication* – Si determinano le comunicazioni necessarie alla esecuzione dei singoli task e le dipendenze tra i task. Occorre individuare le strutture di dati che devono essere passati fra i task.
3. *Agglomeration* – Strutture di comunicazione e task sono valutati rispetto alle performance e costi di implementazione. Se necessario, task differenti sono combinati in macro-task sequenziali allo scopo di ridurre il costo di sviluppo o di performance.
4. *Mapping* – Ogni task o macro-task viene assegnato a un processore con l'obbiettivo di massimizzare l'uso dei processori e minimizzare la comunicazione. Il mapping può essere statico o determinato in run-time da algoritmi di load-balancing



# Partitioning

Lo stage di partitioning è dedicato alla individuazione delle opportunità di esecuzione concorrente per uno specifico algoritmo. L'obbiettivo è definire un ampio numero di piccoli task allo scopo di arrivare a costruire una decomposizione a *grana fine* di un algoritmo di soluzione di un problema. In fasi successive la valutazione dei requirements di comunicazione, l'architettura della macchina destinazione o anche necessità di ingegnerizzazione/implementazione possono poi portare a rivedere l'organizzazione dei task.

Un buon partizionamento prevede contemporaneamente la decomposizione dell'algoritmo in sotto-processi e la decomposizione dei dati su cui l'algoritmo opera. Normalmente prima si determina una appropriata partizione dei dati e successivamente si associa l'elaborazione ai dati stessi. Questo tipo di partizionamento è detto *domain decomposition*. L'approccio alternativo (decomporre prima il calcolo e poi i dati) è detto *functional decomposition*.

In questa fase si evita la *ridondanza* funzionale e nei dati, che può o deve essere però introdotta in una fase successiva per ridurre il costo di comunicazione tra i taski, come per la granularità.

**Domain decomposition**



Nel decomporre i dati associati con un problema si cerca di mantenere i "pezzi" di dimensione pressocché identica. Successivamente si partiziona il calcolo associando chiascuna operazione ai dati sui quali essa deve operare. Tale partizionamento porta a un numero di task piuttosto alto in genere, ognuno dei quali incapsula i dati sui quali opera. Ogni operazione può peraltro richiedere dati prodotti da altri task, ed è quindi richiesta della comunicazione per passare tali dati mutuamente.

I dati da decomporre possono essere di input, di output oppure intermedi. Partizionamenti differenti sono possibili in base a differenti strutture di dati. Normalmente conviene concetrare l'attenzione sulle strutture di dati più grandi.

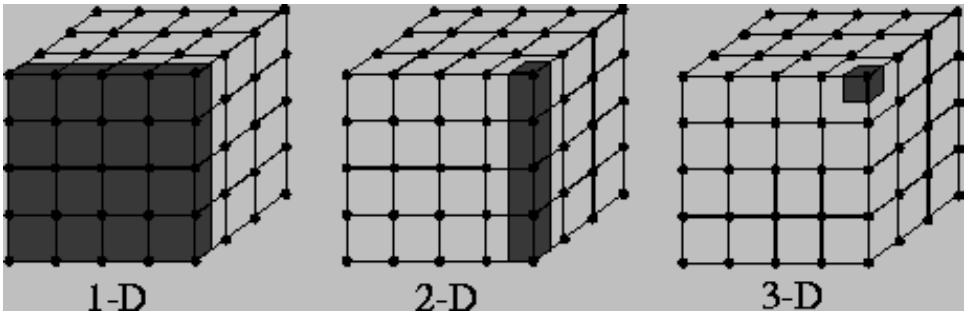



Figura 3: *Decomposizione di dominio per un problema tridimensionale. Sono possibili decomposizioni mono, bi e tridimensionali. Una decomposizione tridimensionale è la più flessibile e può essere adottata nella prima fase del disegno.*

## Functional decomposition

La functional decomposition è complementare rispetto alla domain decomposition. La decomposizione viene effettuata direttamente sul programma, che viene scomposto in task elementari. Successivamente vengono esaminati i dati necessari allo svolgimento di tali task. Tali dati possono essere totalmente disgiunti oppure parzialmente overlapped: in tal caso è richiesta comunicazione. Se la sovrapposizione è cospiscua normalmente è preferibile riconsiderare una domain decomposition.

Sebbene la domain decomposition sia alla base di buona parte degli algoritmi paralleli, concentrare l'attenzione sull'algoritmo può rappresentare un modo per evidenziare possibili caratteristiche di concorrenza non immediatamente visibili dall'analisi dei dati. La decomposizione funzionale inoltre aiuta a modularizzare e strutturare il programma in modo da ridurne la complessità.

Gli esempi precedenti possono essere riguardati alla luce di questo tipo di analisi.



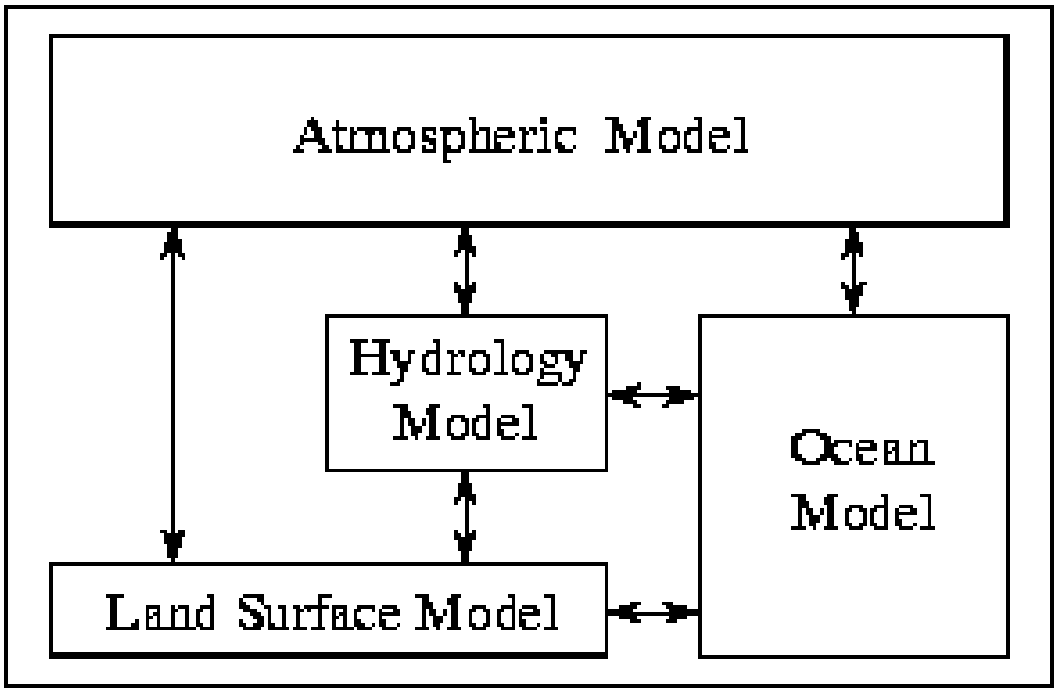



Figura 4: *Decomposizione funzionale per un modello climatologico. Ogni componente è un task che può essere parallelizzato con la decomposizione per dominio.*

## Checklist per il partitioning

Una volta completato il partitioning (che produce uno fra generalmente molte differenti alternative) occorre verificare quanto segue:

1. Il partizionamento ha prodotto un numero di task che è un ordine di grandezza maggiore del numero di processori? Altrimenti ci sarà scarsa flessibilità nelle fasi successive.

2. Il partizionamento ha evitato ridondaza di elaborazione ed eccesso di richieste di dati? Altrimenti ci saranno problemi di scalabilità all'aumentare delle dimensioni di input.
3. I task sono di complessità comparabile? Altrimenti ci sarà disparità nel carico di lavoro per processore.
4. Il numero di task scala con la dimensione del problema? Altrimenti ci saranno problemi di scalabilità?
5. Sono state identificate delle alternative di partizionamento? Si ricordi di verificare la domain e functional decomposition.

**Cosa fare se questi controlli evidenziano la mediocrità del partitioning appena ottenuto?**



# Communication

I task di una partizione devono eseguire parallelamente ma generalmente non indipendentemente. L'elaborazione di un task richiede in genere i dati prodotti da altri e tali dati nel modello task/channel devono essere trasferiti per consentire l'elaborazione. Il flusso infortmativo viene definito nella fase di *comunicazione* del disegno.

Occorre in prima istanza definire i canali (uno per ciascun link informativo necessario fra task distiniti, almeno) e la direzione di comunicazione, quindi le istanze da trasferire. Un modello task channell in genere è adeguato anche per un linguaggio data-parallel.

Generalmente è abbastanza difficoltoso individuare rapidamente le comunicazioni dopo una domain decomposition, e anche semplici decomposizioni possono richiedere complesse strutture di comunicazione.

Viceversa è spesso immediato individuare i requirements di comunicazione dopo una functional decomposition, in quanto corrispondono al normale flusso informativo tra i task.

## Tassonomia delle comunicazioni

- *Locale/Globale* – Un task comunica con pochi vicini o molti/tutti i task;



- *Strutturata/Non strutturata* – I task e i suoi "interlocutori" possono formare una struttura regolare (albero, griglia) oppure un grafo arbitrario;
- *Sincrona/Asincrona* – Produttore e consumatore possono essere coordinati oppure essere indipendenti nella richiesta/fornitura dei dati
- *Statica/Dinamica* – Nella comunicazione dinamica l'identità dei comunicanti cambia nel tempo e potrebbe essere determinta da dati calcolati in runtime $\Rightarrow$ non determinismo.

## Comunicazione locale

È la tipologia di comunicazione che più facilmente si incontra in molti problemi numerici. Normalmente si definisce una task producer e alcuni task consumer per uno o più dati. In genere ogni task assume tale ruolo scambievolmente in tempi diversi.

Per esempio si può considerare un semplice problema numerico come

$$X_{i,j}^{(t+1)} = \frac{4X_{i,j}^{(t)} + X_{i-1,j}^{(t)} + X_{i+1,j}^{(t)} + X_{i,j-1}^{(t)} + X_{i,j+1}^{(t)}}{8}$$



L'insieme di nodi con i quali comunica $X_{i,j}$ è detto *stencil*. Nel caso di domain decomposition viene creato un task per ciascun nodo del reticolo, ciascuno dei quali deve calcolare la successione $\{X_{i,j}^{(t)}\}_t$. Ciascun task definisce dei canali che linkano i 4 vicini prossimi ortogonali e quindi tutti i task hanno la logica seguente

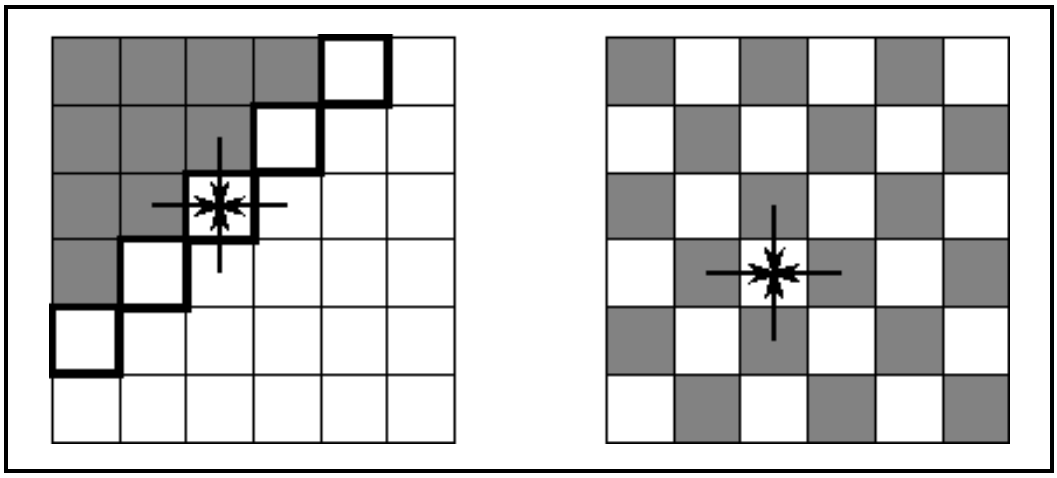
```
for t=0 to T-1 do
   send my_node(t) to neighbors
   receive proxies(my_node) from neighbors
   compute my_node(t+1)
endfor
```

La parallelizzazione è come si può notare "trivial". Nel caso di uno schema Gauss-Seidel del tipo

$$X_{i,j}^{(t+1)} = \frac{4X_{i,j}^{(t)} + X_{i-1,j}^{(t+1)} + X_{i+1,j}^{(t)} + X_{i,j-1}^{(t+1)} + X_{i,j+1}^{(t)}}{8}$$

In tal caso si può procedere con una strategia wave-front oppure red-black.

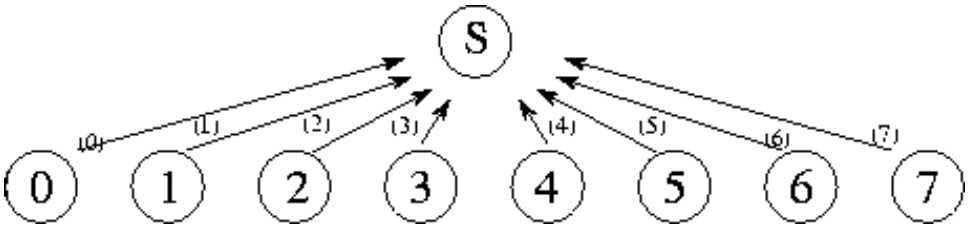




## Comunicazione globale

Nella comunicazione globale molti task devono partecipare e può non essere semplice individuare le coppie producer/consumer. Tale approccio inoltre può ridurre le possibilità di esecuzione concorrente e aumentare considerevolmente la comunicazione. Si consideri ad esempio un problema di *riduzione parallela*

$$S = \sum_{i=0}^{N-1} X_i$$



Supponiamo che un singolo task "manager" debba calcolare $S$, potremmo definire una struttura di comunicazione che consenta a ciascun task associato al nodo $X_i$ di inviare il suo valore indipendentemente. Dal momento che il manager può sommare solo un valore per volta all'accumulatore, questo approccio ha un costo $O(N)$ per completare l'operazione.

Due difetti sono evidenti:

- L'algoritmo è centralizzato, non distribuisce calcolo e comunicazione, in quanto un singolo task deve partecipare ad ogni operazione.
- L'algoritmo è sequenziale, non consente al calcolo di procedere parallelamente alle comunicazioni.

SOLUZIONI?



## Comunicazione strutturata e dinamica

Gli esempi considerati sino ad ora prevedono tutti una comunicazione statica e non strutturata nella quale i task comunicanti formano un pattern regolare (griglia,albero) e non cambiano nel tempo. In altri casi i pattern di comunicazione possono essere considerevolmente più complessi. Un esempio sono i metodi agli elementi finiti per problemi PDE. In tal caso la struttura dati è estremamente irregolare e data-dependent, inoltre può modificarsi nel tempo se la griglia evolve nel tempo.

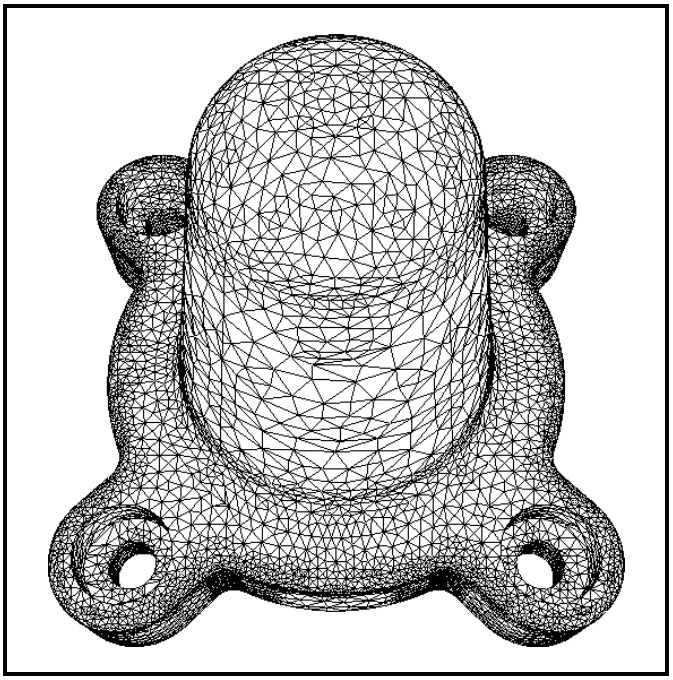


Figura 5: Esempio di dominio per metodo agli elementi finiti



Questo genere di comunicazione non strutturata complica notevolmente le fasi successive di raggruppamento e mapping: algoritmi molto sofisticati possono essere richiesti per definire una strategia di agglomeration che crei task ben bilanciati per costo computazionale e di comunicazione.

La dinamicità della comunicazione costringe inoltre ad applicare tali algoritmi frequentemente durante il running del programma e il costo di tale operazione deve essere aggiunto al costo totale.

**Comunicazione asincrona**

In tale genere di comunicazione i task che possiedono i dati (*producers*) non sono in grado di stabilire quali altri task (*consumers*) richiedono o richiederanno i dati: i consumer in tal caso devono esplicitamente richiedere i dati ai consumer (nel caso di un paradigma message-passing) o prelevarli da una coda (nel caso del paradigma di canale).

Questa situazione è tipicamente presente quando il programma è strutturato in una serie di task che devono accedere periodicamente ad una medesima struttura shared, in scrittura e lettura. Dal momento che tale struttura non è incapsulata in un singolo task è necessario supporta l'accesso concorrente usando un meccanismo tra quelli possibili:



- La struttura è distribuita fra i task. Ogni task genera le richieste per i dati localizzati in altri task ed elabora la propria porzione. Periodicamente effettua un polling delle richieste pendenti per la porzione locale.
- La struttura dati è distribuita in una seconda serie di task il cui compito è solo quello di gestire la comunicazione.
- Su una architettura a SM, i task possono accedere ai dati condivisi senza accorgimenti particolari, tranne per assicurare la sincronizzazione dei dati ovvero l'ordine di lettura/scrittura.

Ogni strategia ha vantaggi e svantaggi e le performance complessive possono cambiare molto a seconda della architettura di calcolatore impiegata. La prima strategia porta a una architettura poco modulare e il polling può essere molto costoso. La seconda strategia è più modulare, ma può essere difficile sfruttare la località in quanto non ci sono dati locali: ogni operazione read/write richiede comunicazione.

## Checklist per la comunicazione

1. I task svolgono tutti la stessa quantità di operazioni di comunicazione? La comunicazione non bilanciata porta poca scalabilità. Può essere opportuno replicare i dati ad accesso frequente o fare un merge di certi task.



2. I task comunicano ciascuno con un numero ridotto di primi vicini? Se così non fosse può essere il caso di esplorare strategie alternative di distribuzione dei dati, come nell'algoritmo di interazione a coppie.
3. Le operazioni di comunicazione sono concorrenti? Se così non fosse il programma non sarà scalabile. Si deve provare a implementare una strategia dividi-conquista come nell'algoritmo di sommatoria.
4. Il calcolo può essere svolto in concorrenza in massima parte? Diversamente il programma non sarà scalabile e sarà molto inefficiente. Si consideri la possibilità di riordinare comunicazione e calcolo o rivedere gli algoritmi di base.



# Raggruppamento (Agglomeration)

L'algoritmo ricavato dalle prime due fasi è ancora "generico" ovvero non specializzato per una esecuzione efficiente su nessun computer parallelo. In tale fase occorre rivedere le decisioni prese, allo scopo di raggruppare dei task identificati nella fase di partitioning ed eventualmente replicare calcolo o dati fra i task.

Il risultato di tale raggruppamento potrebbe ancora essere un numero di task troppo elevato per il numero di processori disponibili, per cui rimarrebbe ancora non specificato il mapping dei task sui processori. In alternativa, già in tale fase si può scegliere di ridursi a un numero di task pari al numero di processori. Nel caso di problemi SPMD il disegno in tale fase è completo e può essere direttamente specificato il mapping.

*Obbiettivi guida: ridurre le comunicazioni incrementando il calcolo e la grnaularità di comunicazione; mantenere la flessibilità rispetto a a decisioni di mapping e scalabilità ulteriori; ridurre i costi di ingegnerizzazione.*

## Aumentare la granularità

Nel partizionamento si è massimizzato il numero di task, ma questo in generale non produce un programma efficiente



a causa dei costi di comunicazione. Per ridurre la comunicazione si possono ridurre i canali oppure la quantità di dati trasmessi. Aumentare la granularità dei dati consente appunti di ridurre il quantitativo di dati trasmessi per canale.

Un ulteriore costo da considerare è quello di creazione dei task, soprattutto quando il numero di task è estremamente elevato.

Se il numero di partner di comunicazione per task è basso, si può spesso ridurre il numero di operazioni e il volume di comunicazione totale raggruppando i task ovvero aumentando la grana di partizionamento ($\Rightarrow$ *surface–volume*).

Il rapporto *comunicazione/calcolo* decresce al crescere delle dimensioni del task e tale effetto è spesso marcatamente visibile quando il partizionamento è di tipo domain guided.

Il raggruppamento può essere difficoltoso quando la comunicazione è non strutturata.

**Replicare i dati**

Soprattutto su specifiche architetture operazioni di broadcast o comunque la replica dei dati consente di minimizzare i costi di comunicazione complessivi. Un esempio è il *butterfly* per il calcolo della sommatoria. Questa struttura consente



di evitare il broadcast del risultato a tutto l'albero per continuare l'elaborazione e svolge il calcolo in $O(log(N)$.

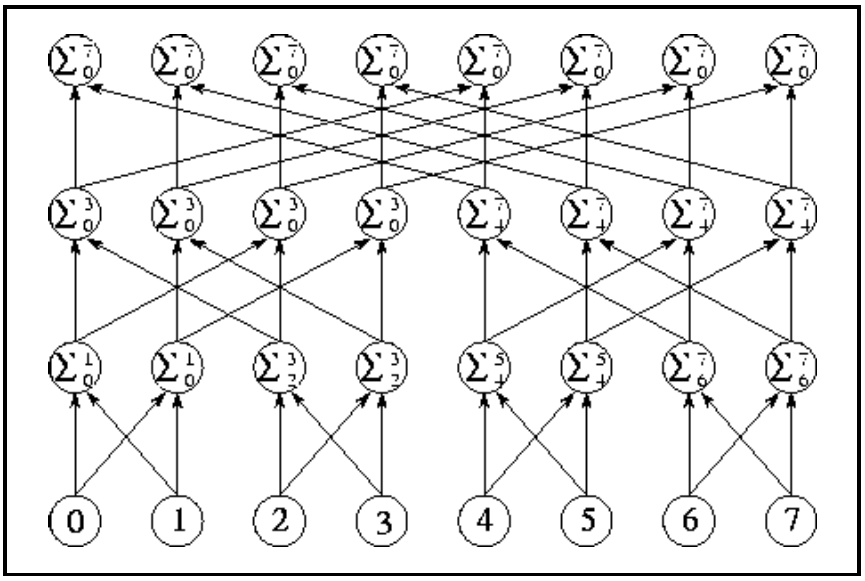



Figura 6: *Butterfly per il calcolo della sommatoria* $\sum_0^7$

In ogni stage ogni task riceve dati a due task, svolge una singola addizione e invia il risultato a due task dello stage successivo. Il raggruppamento è quasi sempre una soluzione quando l'analisi della comunicazione rivela che un insieme di task non può essere svolto in concorrenza. Considerando le strutture ad albero solo i task allo stesso livello possono essere svolti in concorrenza, per cui a meno di implementare un pipe-line (richiesto dal problema) i task a differenti livelli possono essere raggruppati senza perdere concorrenza e minimizzando la comunicazione.

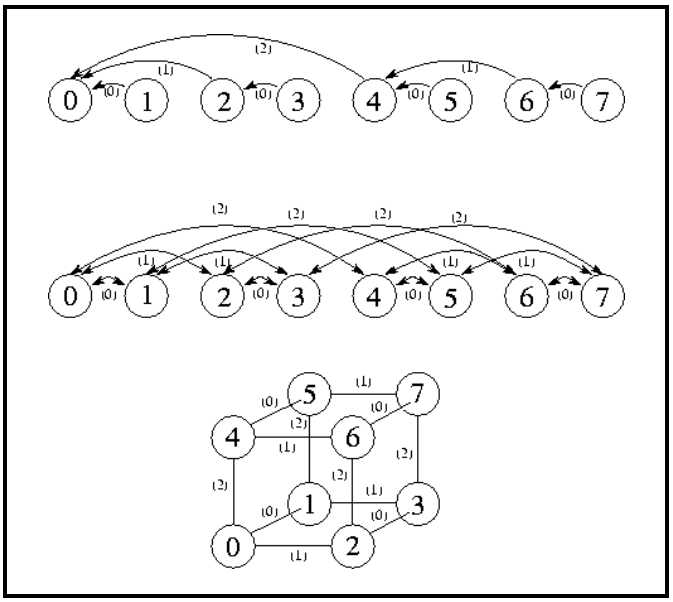



Figura 7: *Comunicazioni risultanti quando vengono raggruppati task a diversi livelli di un albero o di una butterfly. In ogni caso il canale è labellato con lo step nel quale viene utilizzato.*

## Mantenere la flessibilità

Facilmente quando si raggruppa si può compremettere la scalabilità, ad esempio decomponendo una struttura di dati multidimensionale in una sola dimensione: questo tipo di strategia non sarebbe adatta a sistemi a parallelismo massiccio (come le macchine SIMD).

La possibilità di creare un numero di task variabili è critica per la scalabilità e portabilità di un programma. La flessibilità potrebbe tornare comoda in certi casi: ad esempio se

un gruppo di task si bloccano spesso in attesa di dati remoti potrebbe essere opportuno mapparli tutti sullo stesso processore perchè questo minimizza i tempi di idle del processore stesso.

Si parla in tal caso di *overlapping communication and computation*

Ulteriori benefici possono derivare in fase di mapping, in quanto un numero di task elevati consente l'esplorazione di strategie multiple di loading e bilanciamento. Una regola generale è che il numero di processi dovrebbe essere di un ordine di grandezza superiore rispetto al numero di processori disponibili.

*Il disegno non deve incorporare limitazioni non necessarie sul numero di task da creare, in quanto la grana dovrebbe essere se possibile definita in run-time.*

## Ridurre i costi di ingegnerizzazione

Se possibile si dovrebbe considerare il costo di sviluppo legato alle diverse strategie di partizionamento e comunicazione, cosa particolarmente importante se si deve parallelizzare codice sequenziale pre-esistente. Da questo punto di vista può essere opportuno adottare partizionamenti che riducano



al minimo la necessità di modificare il codice e algoritmi impiegati nella versione sequenziale: se il codice opera su una griglia multidimensionale non è opportuno partizionare in una sola dimensione.

Il problema del costo di ingegnerizzazione non è banale soprattutto per programmi di grandi dimensioni, dove ogni approccio di parallelizzazione ha a priori caratteristiche di performance.

## Checklist per l'agglomeration

Alcuni dei criteri da considerare sono quantitativi, e questo aspetto diventa man mano più importante man mano che si avanza nel disegno della applicazione.

1. Il raggruppamento ha ridotto la comunicazione e aumentato la località?
2. Se il raggruppamento ha replicato il calcolo, si è verificato che il benefici di tale ridondanza sopravanzano il suo costo, per diverse dimensioni del problema e numero di processori?
3. Il raggruppamento ha portato a task con costi di calcolo e comunicazione bilanciati? Più "grandi" sono i task prodotti e più è importante che abbiano costi simili.



4. Il numero di task è scalabile con il numero di processori? Altrimenti l'algoritmo non sarà portabile su sistemi a parallelismo massiccio o adatto a risolvere problemi di grandi dimensioni.
5. Se il raggruppamento ha eliminato opportunità di esecuzione concorrente, si è verificato che ci sia ancora un grado di concorrenza elevato, indipendentemente dalla architettura di macchina da impiegare? Modelli di performance potrebbero essere impiegati per stimare i tradeoff complessivi.
6. Il numero di task può essere ulteriormente ridotto, senza introdurre sbilanciamento o ridurre la scalabilità? Algoritmi che creano pochi task a grana larga sono generalmente più semplici ed efficienti di altri che creano molti task a grana fine.
7. Se si sta parallelizzando un programma sequenziale pre-esistente, si sono considerati i costi di re-ingegnerizzazione? Se tali costi sono elevati si considerino alternative di raggruppamento differenti per aumentare le opportunità di riutilizzo del codice. Se l'algoritmo risultante fosse meno efficiente, si usino tecniche di performance modeling per determinare il costo di tradeoff.



# Mapping

Nello stage finale del disegno di un programma parallelo, si specifica su quale processore ogni singolo task deve essere eseguito. Tale mapping non si applica nel caso uniprocessore o per sistemi a shared-memory che tipicamente hanno già uno scheduling automatico dei processi.

Algoritmi general-purpose per il mapping devono ancora essere sviluppati, il che fa intuire quanto il problema sia a priori complesso.

L'obbiettivo del mapping è la minimizzazione del tempo totale di esecuzione e questo si può ottenere con due strategie:

- I task che possono essere svolti in concorrenza devono essere eseguiti su processori differenti (massimizzazione della concorrenza).
- I taks che comunicano frequentemente devono essere eseguiti sullo stesso processore (massimizzazione della località).

Limiti pratici sono normalmente da considerare per il numero di processi che possono essere mappati efficientemente sullo stesso processore. Il problema del mapping è notoriamente



un *problema NP-completo*, tuttavia una certa quantità di strategie euristiche è stata individuata per architetture di programmi tipiche.

Molti algoritmi sviluppati da una domain decomposition hanno un numero fissato di task omogenei con comunicazione locale e globale strutturata e bilanciata. In tali casi un mapping efficiente è immediato: i task vengono mappati in modo da minimizzare l'interprocess communication; se non è stato già fatto si raggruppano i processori in un numero di task paro al numero di processori.

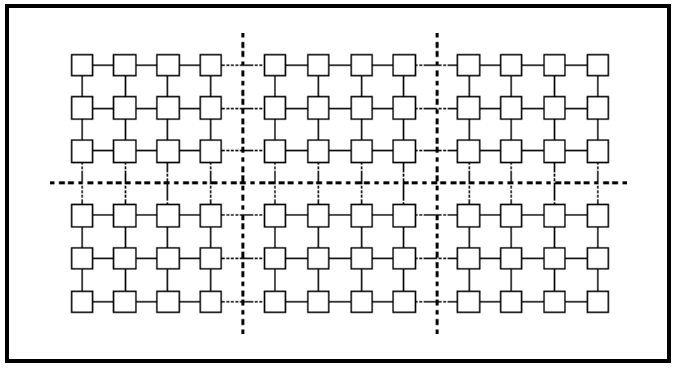


Figura 8: *Mapping nel caso di task bilanciati per computation e communication.*

In problemi più complessi nei quali il costo computazionale per task non è bilanciato e/o sono presenti pattern di comunicazione non strutturati, strategie di mapping e raggruppamento efficienti sono difficoltose da individuare. Si

possono impiegare degli algoritmi di *load balancing* basati tipicamente su tecniche euristiche Il tempo di esecuzione di tali algoritmi deve essere pesato a sua volta rispetto ai benefici sul tempo di esecuzione totale. Metodi probabilistici hanno un carico inferiore normalmente rispetto a metodi che analizzano la struttura della applicazione.

I problemi più complessi sono quelli cge hanno un numero di task e/o un carico computazionale e/o di comunicazione che varia in corso di esecuzione. Nel caso di problemi ricavati da domain decomposition si può utilizzare una strategia di *dynamic load balancing* nella quale gli algoritmi precedentemente indicati vengono eseguiti periodicamente per ridefinire raggruppamento e mapping: in tal caso algoritmi locali sono da preferire.

I programmi basati su decomposizione funzionale spesso sono strutturati in molti short-living task cge su coordinano fra loro: in tali casi si possono utilizzare algoritmi di task-scheduling, che allocano i task sui processori disponibili prelevandoli da una o più code di richieste.

## Algoritmi di load-balancing

Quelli che seguono sono algoritmi di parizionamento, il cui scopo è raggruppare task a grana fine rispetto a un numero



di processori a grana maggiore.

**Recursive bisection** È una tecnica di partizionamento che quando eseguita on fly consente una implementazione parallela. Consiste nel partizionare il dominio in parti uguali lungo le dimensioni del dominio stesso e nel riapplicare la stessa tecnica ricorsivamente in ogni sezione. Le tipologie di bisezione principali sono

**recursive coordinate bisection** Il sezionamento avviene lungo la dimensione più lunga. Particolarmente utile in caso di domini blandamente irregolari.

**unbalanced recursive bisection** Si cosidera il bilanciamento del costo partizionando le $P-1$ partizioni ottenute formando sottogriglie non bilanciate con carichi $\frac{1}{P}$ e $\frac{(P-1)}{P}$, $\frac{2}{P}$ e $\frac{(P-2)}{P}$ e così via scegliendo la partizione che minimizza l'aspect ratio. Questo sistema aumenta il costo di partizionamento, ma può ridurre il costo di comunicazione.

**recursive graph bisection** Utile in caso di griglie irregolari: si identificano prima i due estremi del grafo, cioè i due vertici che sono più distanti in termini di graph distance. Ogni vertice viene assegnato a sottodomini corrispondenti alle estremità più vicine. Anche in questo caso si procede ricorsivamente. Lo scopo è minimizzare il costo di comunicazione



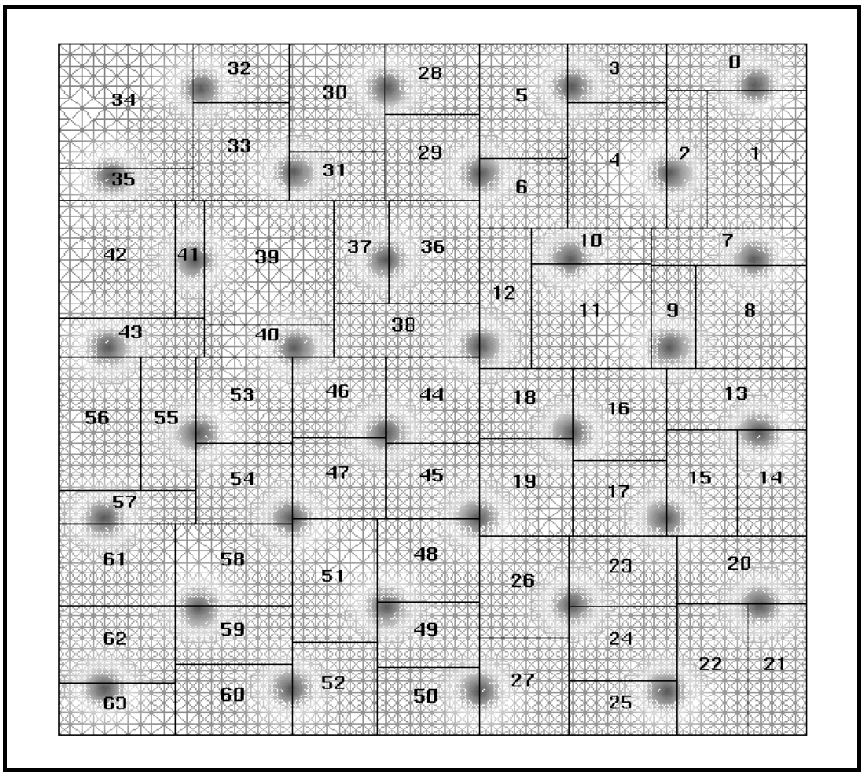



Figura 9: *Unbalanced recursive bisection*

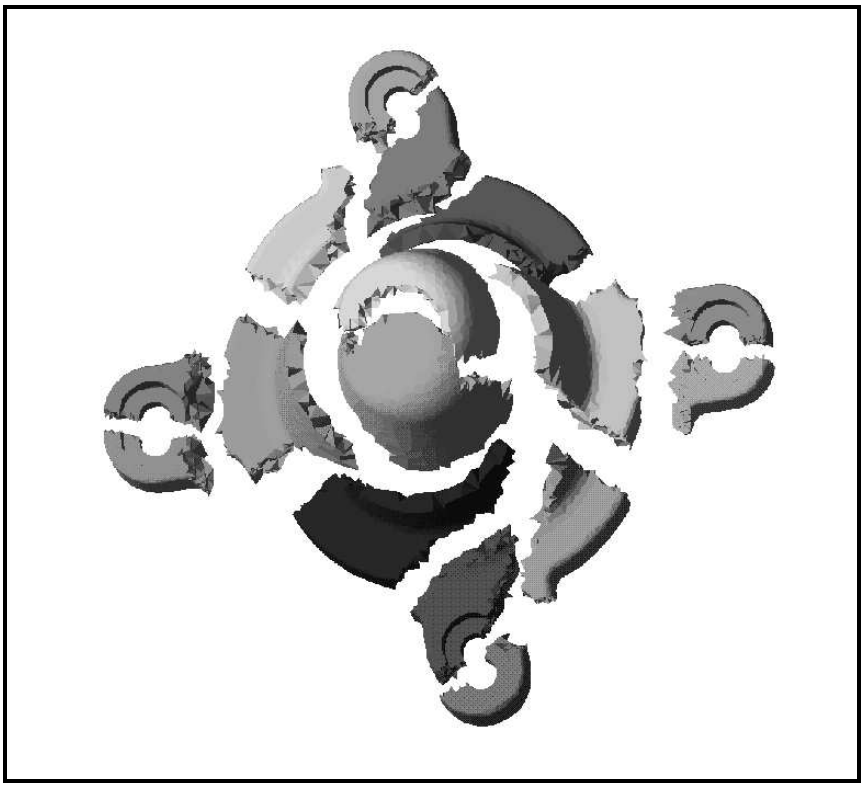


Figura 10: *Graph recursive bisection*

**Algoritmi locali** Le tecniche precedenti sono costose in quanto richiedono conoscenze globali sullo stato computazionale. Le tecniche locali cercano di limitarsi a informazioni ottenute solo da un numero limitato di processori vicini, as esempio i processori possono essere organizzati in un reticolo e ogni processore può periodicamente confrontare il proprio carico con quello dei vicinie trasferire processi se la differenza supera una data soglia.

**Metodi probabilistici** Una tecnica immediata è assegnare i

processi ai processori in modo random. Se il numero di task è elevato ci si può aspettare che il carico computazionale sia uniforme. Il risultato atteso però è in media un maggiore costo medio off-processor di comunicazione

**Mapping ciclici** Una variante del mapping probabilistico. In tal caso si introduce una enumerazione (basata su località) dei processi e si allocano i processori ai processi seguendo la numerazione dei task. L'obiettivo è cercare di mantenere basso l'accoppiamento sui dati inter-processore

## Algoritmi di task-scheduling

Questo genere di algoritmi sono utili nel caso di decomposizione funzionale, che porta normalmente a molti task, ciascuno con deboli vincoli di località.

L'aspetto più complesso di un algoritmo di task-scheduling è la strategia da adottare per allocare i problemi ai worker: tale strategia deve essere un compromesso fra la riduzione della comunicazione (favorendo l'indipendenza funzionale) e la massimizzazione del bilanciamento (aumentando la conoscenza dello stato computazionale globale).

**Master e slaves** Utilizzabile efficacemente per un piccolo numero di processori. Il manager è responsabile anche



delle allocazioni dei processori. Ciascun worker può anche richiedere la allocazione di nuovi task al manager. L'efficienza può essere aumentata con tecniche di pre-fetching dei task/dati in modo da sovrapporre calcolo e comunicazione, oppure "cachando" i task nei worker, in modo che il manager sia costretto a comunicare solo quando nessun task è gia' disponibile.

**Master e slaves gerarchico** Variazione del precedente che divide i task in insiemi disgiunti, ognuno con un sub-manager. La comunicazione fra i submanager deve bilanciare il carico tra gli insiemi di processori di cui ogni sub-manager è responsabile.

**Schemi decentralizzati** Ogni processore mantiene un pool di task e i processori in "idle" richidono i task ai propri "vicini" oppure in modo random a uno o più dei worker vicini. La maggiore scalabilità si paga in termini di maggiore coplessità della architettura del programma. Calcolo e task pool management possono inoltre essere associati agli stessi task oppure a task diversificati.

**Detection di terminazione** Occorre un criterio di stop per definire quando deve cessare l'allocazione dei processori. Tale criterio di stop è facilmente determinato nel caso di management centralizzato, molto più complesso nel caso di schemi decentralizzati.



# Checklist per il mapping

1. Nel caso di programmi SPMD, sono stati considerati algoritmi basati sulla creazione/distruzione dinamica di task? Occhio alle performance.
2. Viceversa?
3. Usando uno schema master/slave, è sicuro che non si crei un collo di bottiglia nelle comuicazioni? Si può ridurre la comunicazione passando descrittori dei task piuttosto dei task medesimi al manager.
4. In caso di bilanciamento dinamico, sono stati valutati i costi di differenti strategie di bilanciamento? Schemi ciclici o probabilistici vanno sempre considerati perchè più semplici e richiedono meno operazioni di ri-bilanciamento.
5. Per schemi dinamici, il numero di task è sufficientemente elevato da garantire un ragionevole bilanciamento? Tipicamente i task dovrebbero essere almeno 10 volte il numero di processori disponibili.



# Analisi di performance

Lo studio analitico delle performance di un algoritmo parallelo ha varie motivazioni e complessità a seconda delle metriche che si è interessati a utilizzare (tempo di esecuzione,efficienza,occupazione di memoria,throughput,latenza,I/O rate,network throughput,costi di analisi e disegno,portabilità ...).

In questa sezione vedremo solo due aspetti, il tempo di esecuzione e la scalabilità parallela, in quanto sono le meglio formalizzate e le più problematiche da definire.

## Approcci alla modellazione delle performance

Vediamo tre tecniche utilizzate per caratterizzare le performance di algoritmi paralleli. Vedremo perchè ciascuna di esse è *inadeguata* a una misura di performance.

**Legge di Amdahl** Una osservazione comune è il fatto che ogni algoritmo ha una componente sequenziale che può limitare lo speedup raggiungibile con una implementazione parallela. i*Lo speedup è il rapporto tra tempo di esecuzione su un singolo processore e tempo di esecuzione su più processori*. Tale osservazione è spessa codificata come *legge di Amdahli*: se la componente sequenziale



di un algoritmo prende una frazione $1/s$ del tempo di esecuzione complessivo, allora il massimo speedup è $s$. Per esempio se la componente sequenziale è un 5% allora il massimo speedup è 20.

In realtà tale osservazione è fondamentalmente fuorviante. Quasi tutti i problemi computazionali ammettono una soluzione parallela. Tale soluzione può essere poco scalabile, ma tale limite è legato a costi di comunicazione, idle time, ridondanza nel calcolo piuttosto che alla esistenza di "componenti sequenziali".

La legge di Amdahl può essere rilevante se la parallelizzazione è fatta in modo incrementale, in quanto il costo computazionale delle componenti non parallelizzate effettivamente limita inferiormente il tempo di esecuzione totale: di fatto tale parallelizzazione parziale è efficace solo su piccoli sistemi. Inoltre la legge può essere utile nella analisi di programmi data-parallel nelle quali alcuni componenti possono non essere suscettibili di una formulazione data-parallel.

**Estrapolazione da osservazioni** Spesso le performance di un algoritmo parallelo vengono descritte come segue: "Abbiamo inplementato l'algoritmo su un computer parallelo X e raggiunto uno speedup di 10.8 su 12 processori con una dimensione di input $N = 100$". Tale misura puntuale vorrebbe rappresentare una misura di perfor-



mance, in realtà è una rappresentazione estremamente superficiale.

Le equazioni seguenti mostrano come sia limitante basarsi sulle osservazioni come tool per misurare le performance di un algoritmo parallelo. Ogni funzione è un modello di performance che specifica il tempo di esecuzione $T$ in funzione del numero di processori $P$ e della dimensione del problema $N$. In ciascun caso assumiamo che la versione sequenziale abbia un costo computazionale $O(N + N^2)$.

- $T = N + N^2/P$ Questo algoritmo partiziona la componente che richiede un $O(N^2)$ e replica il la componente che richiede un $O(N)$ su ciascun processore. Non ci sono altre sorgenti di overhead.
- $T = (N + N^2)/P + 100$ Questo algoritmo partiziona tutto il calcolo ma aggiunge un overhead.
- $T = (N + N^2)/P + 0.6P^2$ Anche questo algoritmo partiziona tutto il calcolo, ma aggiunge un overhead che è funzione del numero di processori.

Questi algoritmi raggiungono tutti un overhead di 10.8 quando $P = 12$ e $N = 100$, ma si comportano ben diversamente in altre situazioni



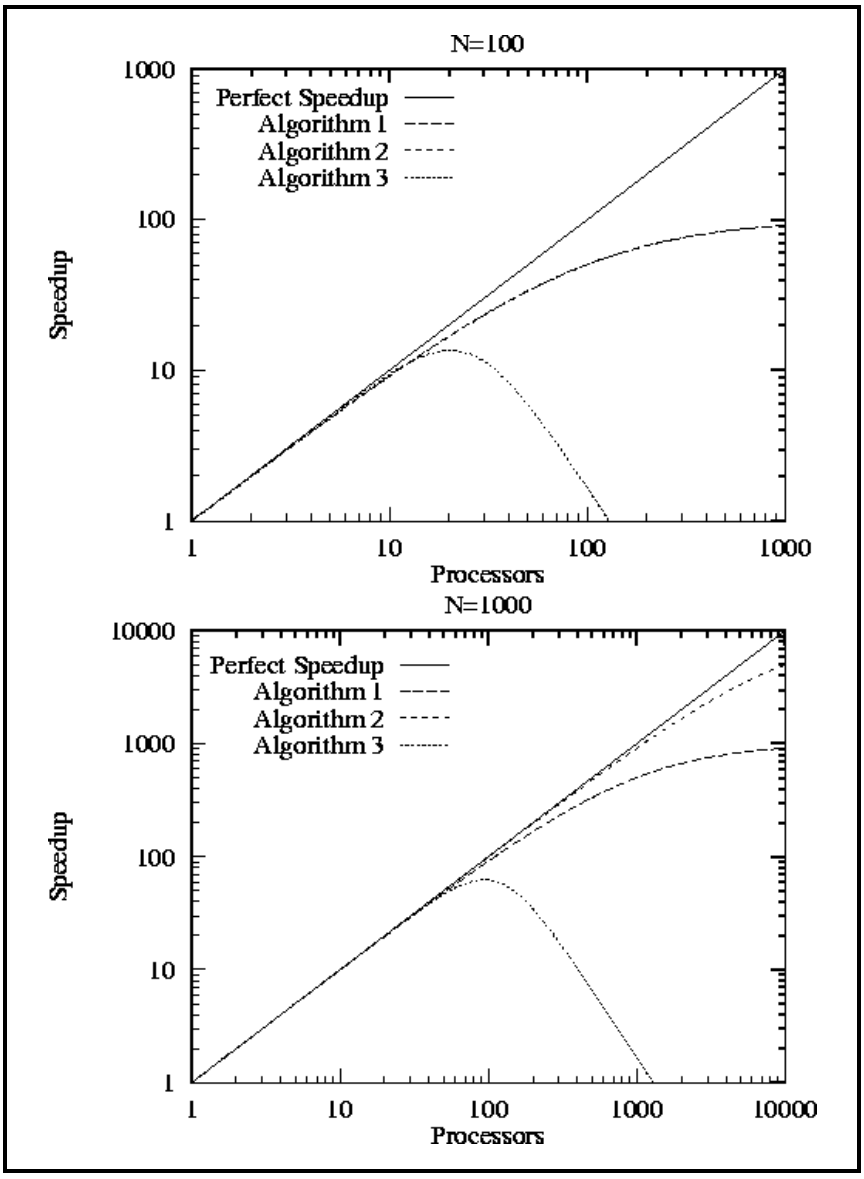



Figura 11: *Speedup in funzione di $P$ per tre differenti algoritmi. Si noti l'uso della scala logaritmica.*

**Analisi asintotica** Spesso viene caratterizzata l'efficienza di una implementazione parallela in questo modo: "L'analisi

asintotica rivela che questo algoritmo richiede un tempo $O(NlogN)$ su $O(N)$ processori". Ovvero esiste una costante $c$ e una dimensione di problema minima $N_0$ tale che per $N > N_0$ il costo è minore di $cN\log(N)$ u $N$ processori. Tale relazione dice come il costo varia con $N$ quando $N$ e $P$ sono grandi.

Sebbene tale informazione sia interessante, spesso non è direttamente rilevante per lo sviluppo di un algoritmo parallelo efficiente. Valendo per $N$ e $P$ grandi difatto ignora i termini di ordine minore che possono essere significativi per problemi di dimensioni pratica e numero di processori ridotto. Ad esempio un algoritmo con complessità computazionale $N\log(N)$ potrebbe essere del tipo $10N + N\log(N)$.

Un secondo limite è il fatto che l'analisi asintotica non dice nulla del costo assoluto. L'analis asintotica suggerisce che un algoritmo con costo $1000N\log(N)$ è superiore a un algoritmo con costo $10N^2$

Infine l'analisi asintotica si basa su un modello di macchina idealizzata che è molto diversa da una qualsiasi macchina "fisica" e non tiene conto di tutti i costi accessori.

## Sviluppare un modello di analisi



Un modello dui performance deve essere in grado di spiegare le osservazioni sperimentali e fornire una previsione, astraendosi da dettagli non significativi. I tre criteri precedenti non sono in grado di soddisfare tale richiesta. D'altronde tecniche di modellazione di sistemi convenzionali dipendono da troppi dettagli sull'hardware per essere praticamente impiegabili sui sistemi paralleli. Nel seguito introduciamo una tecnica di modellazione di livello intermedio, adatta ad architetture di multicomputer.

La metrica impiegata è il tempo di esecuzione $T$ come funzione della dimensione del problema $N$, del numero di processori $P$, del numero di task $U$ e di altre caratteristiche (variabili) dell'algoritmo de dell'hardware.

$$T = f(N, P, U, \ldots)$$

Definiamo tempo di esecuzione come il tempo che trascorre tra il momento in cui il prio processore inizia l'elaborazione e il momento in cui l'ultimo processore termina l'esecuzione. Tale definizione non è completamente adeguata su sistemi time–sharing. Durante l'esecuzione ogni processore calcola, comunica o è in idle per cui

$$T = T^j_{\mathrm{comp}} + T^j_{\mathrm{comm}} + T^j_{\mathrm{idle}}$$



oppure anche

$$T = \frac{1}{P}(T_{\text{comp}} + T_{\text{comm}} + T_{\text{idle}}) =$$
$$\frac{1}{P}(\sum_j T^j_{\text{comp}} + \sum_j T^j_{\text{comm}} + \sum_j T^j_{\text{idle}})$$

L'obiettivo è allora sviluppare delle espressioni matematiche che permettano di specificare il tempo di esecuzione come funzione di $N, P, \ldots$. Per ridurre la complessità del problema si usano le tecniche sequenti:

- Basare il modello su una architettura di multicomputer. I dettagli sull'hw di basso livello quali cache, gerarchie di memorie e topologie di interconnessione vengono introdotti solo se vi è una evidenza sperimentale che suggerisce di operare in tal senso.
- Usare l'*analisi in scala* per identificare sezioni di programma non significative che possono essere ignorate in analisi. Ad esempio se un algoritmo consiste in una fase di inizializzazione seguita da migliaia di iterazioni di calcolo, a meno che l'inizializzazione sia molto costosa, si considerano solo le iterazioni in analisi.
- Usare *studi empirici* per calibrare modelli campione piuttosto di sviluppare da zero modelli più complessi.



# Tempi di esecuzione

**computation time** Avendo una versione sequenziale dell'algoritmo si può dedurre da questa il tempo di calcolo. Altrimenti occorre sviluppare delle componenti chiave (da individuare) per effettuare il calcolo. Il tempo dipende normalmente dalla misura del problema, che può essere rappresentato da un singolo parametro $N$ o da un vettore di parametri $\{N_k\}$. Se l'algoritmo replica il calcolo il tempo dipende dal numero di task o processori. Su architetture eterogenee il tempo dipende dal tipo di processore/host su cui viene eseguito il task.

Non si può assumere che il tempo rimanga costante o vari linearmente quando il numero di processori o la dimensione del problema aumenta.

**communication time** Possono essere distinti un tempo di comunicazione interprocessore e uno intraprocessore. Per semplicità si suppone che i due costi siano comparabili. Tale assunzione è ragionevole su molti multicomputer a meno che la comunicazione intraprocessore non sia altamente ottimizzata, questo perchè il costo delle copie memory-to-memory e i context switch sono spesso comparabili al costo di comunicazione interprocessore. In alcune architetture questo non è vero, ad esempio reti di workstation connesse via rete.



In una architettura ideale il costo di invio di un messaggio fra task in run su processori diversi può essere rappresentato da due parametri: i*startup time* e *transfer time per word*

$$T_{\mathrm{msg}} = t_s + t_w L$$

Chiaramete tali tempi sono legati alla architettura fisica del multicomputer.

| *Machine* | $t_s$ | $t_w$ |
|---|---|---|
| IBM SP2 | 40 | 0.11 |
| Intel DELTA | 77 | 0.54 |
| Intel Paragon | 121 | 0.07 |
| Meiko CS-2 | 87 | 0.08 |
| nCUBE-2 | 154 | 2.4 |
| CM5 | 82 | 0.44 |
| Ethernet | 1500 | 5.0 |
| FDDI | 1150 | 1.1 |

Tabella 1: *Parametri temporali in $\mu$sec per varie architetture di macchine*



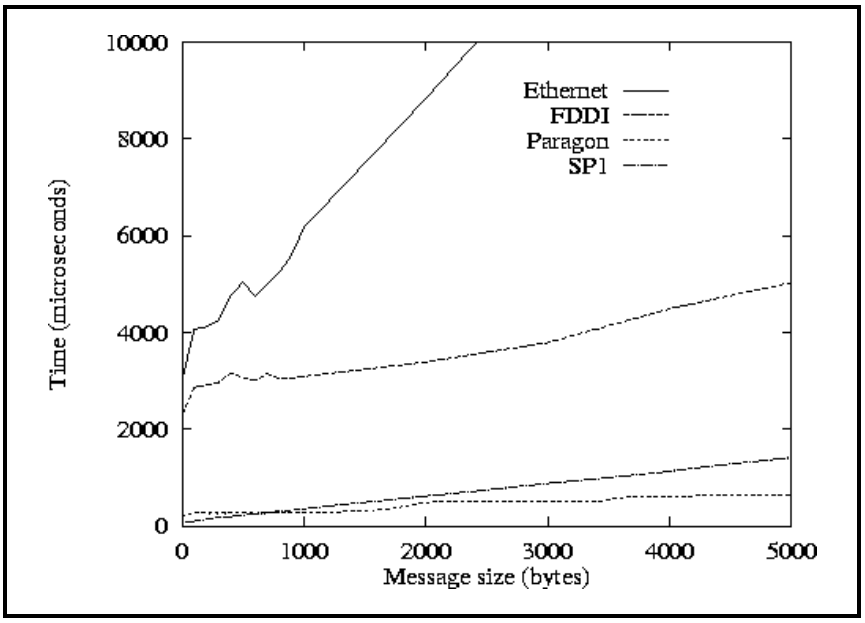



Figura 12: *Round-trip time per un messaggio di dimensioni variabili su varie architetture.*

Le irregolarità nei tempi in funzione della lunghezza del messaggio sono dovute alle implementazioni reali delle interfacce e librerie di controllo del canale fisico di comunicazione. In ogni caso il modello lineare indicato è in genere una sufficiente approssimazione.

Applicazioni con comunicazione non strutturata o meno regolare in generale avrà performance inferiori. Costi addizionali quali il management di buffer e traduzioni di dati (cfr XDR) non sono inclusi, ma sono tipicamente proporzionali alla lunghezza del messaggio. È possibile fittando l'equazione lineare ai dati reali ottenere i parametri temporali in dipendenza del sistema e dell'algoritmo

specifico.

Bisogna considerare i problemi pratici legati alla valutazione dei tempi nel fare tali misure.

**idle time** Questi tempi sono complicati da valutare perchè spesso dipendono dall'ordine di esecuzione delle operazioni.

Un processore può essere in idle per mancanza di dati o per mancanza di istruzioni da eseguire. Nel secondo caso è opportuno implementare algoritmi di load-balancing o scheduling. Nel primo caso i dati possono essere in elaborazione su processori remoti. Tale tempo può essere ridotto strutturando il programma in modo che il task in attesa di dati svolga altri compiti nel frattempo (overlapping di comunicazione e calcolo).

Si può creare task multipli su ciascun processore in modo che un task possa elaborare mentre altri sono in attesa di dati. Tale approccio è ragionevole solo se il costo di scheduling è minore del costo di idle time. Diversamente si può implementare un task con l'interleaving esplicito di calcolo e comunicazione (per esermpio usando programmazione multi–thread).

## Esempio: differenze finite

Si consideri per semplicità una griglia $N \times N \times Z$ dove



$Z$ per un problema alle differenze finite tridimensionale. Inizialmente supponiamo che la griglia sia decomposta lungo una sola dimensione partizionata fra $P$ tasks. Ciascuno di essi è responsabile di una sotto-griglia di dimensione $N \times (\frac{N}{P}) \times Z$.

$$T_{\text{comp}} = t_c N^2 Z_j$$

dove $t_c$ è il tempo medio di calcolo su ciascun punto della griglia. Si considera uno stencil di 9 punti, ovvero ogni task deve scambiare $2NZ$ dati con i suoi vicini per un totale di due messaggi di dimensione $4NZ$ (assumiamo che ogni processore è allocato per almeno una griglia $2N$. Il costo sommato su $P$ processori è allora:

$$T_{\text{comm}} = 2P(t_s + t_w 2NZ)$$

Supponendo sia lecito trascurare l'idle time si ottiene

$$\begin{aligned} T_{\text{P}} &= \frac{T_{\text{comp}} + T_{\text{comm}}}{P} \\ &= t_c \frac{N^2 Z}{P} + 2t_s + t_w 4NZ \end{aligned}$$

Altre misure di performance utili per misurare le performance relative della architettura sequenziale rispetto alle performance di una versione sequenziale:



## Efficienza relativa

$$E_{\text{rel}} = \frac{T_1}{PT_P}$$

## Speedup relativo

$$S_{\text{rel}} = PE_1$$

dove per *efficienza* si intende il rapporto $\frac{T_{\text{comp}}}{T_{\text{tot}}}$

Nel caso delle differenze finite, l'efficienza assoluta si misura come

$$E = \frac{t_c N^2 Z}{t_c N^2 Z + t_s 2P + t_w 4NZP}$$



# Analisi di scalabilità

Il modelli di performance possono essere usati per esplorare e rifinire il disegno di un algoritmo parallelo. Tali modelli possono anche essere impiegati per svolgere una analisi qualitativa delle performance, per esempio nel caso del problema modello precedente:

- L'efficienza descresce al crescere di $P, t_s \mathrm{e} t_w$
- L'efficienza cresce al crescere di $N, Z e t_c$
- Il tempo di esecuzione decrescere al crescere di $P$ ma è limitato inferiormente dal costo di trasmissione.
- It tempo di esecuzione cresce al crescere di $N, Z, t_c, t_s \mathrm{e} t_w$

L'uso di dati empirici (machine-specific) rilevati come mostrato in precedenza può inoltre facilitare la risposta ai quesiti seguenti:

- L'algoritmo incontra i vincoli di disegno (relativamente a tempi di esecuzione, richiesta di memoria, etc) sul computer parallelo da utilizzare per l'implementazione?
- Quanto è "adattabile" l'algoritmo? Ovvero quanto si adatta all'aumentare delle dimensioni del problema e del numero di processori? Quanto è sensibile al modificarsi dei parametri $t_s$ e $t_w$?



- Come si confronta l'algoritmo con altri che risolvono lo stesso problema? Che differenza temporale ci si può aspettare da differenti algoritmi?

## Scalabilità con dimensione di input fissata

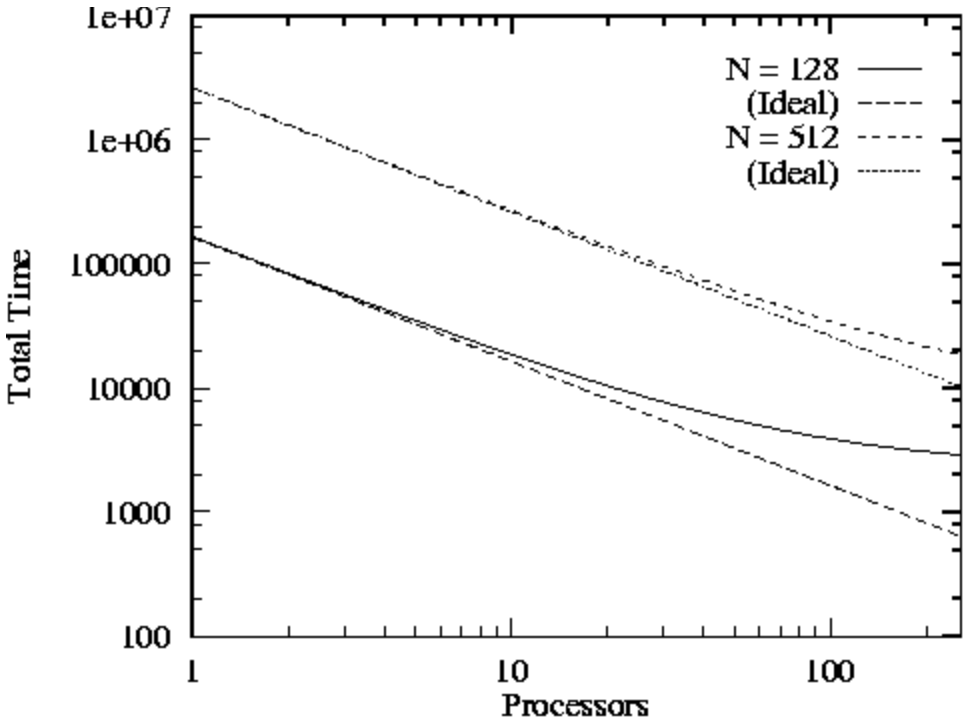
Un aspetto importante è l'analisi delle performance al variare dei parametri del programma. In particolare si può valutare la scalabilità ovvero le performance al variare del numero di processori. La tecnica consiste nel misurare $T$ e $E$ in funzione di $P$ fissati i parametri macchina e la dimensione del problema.

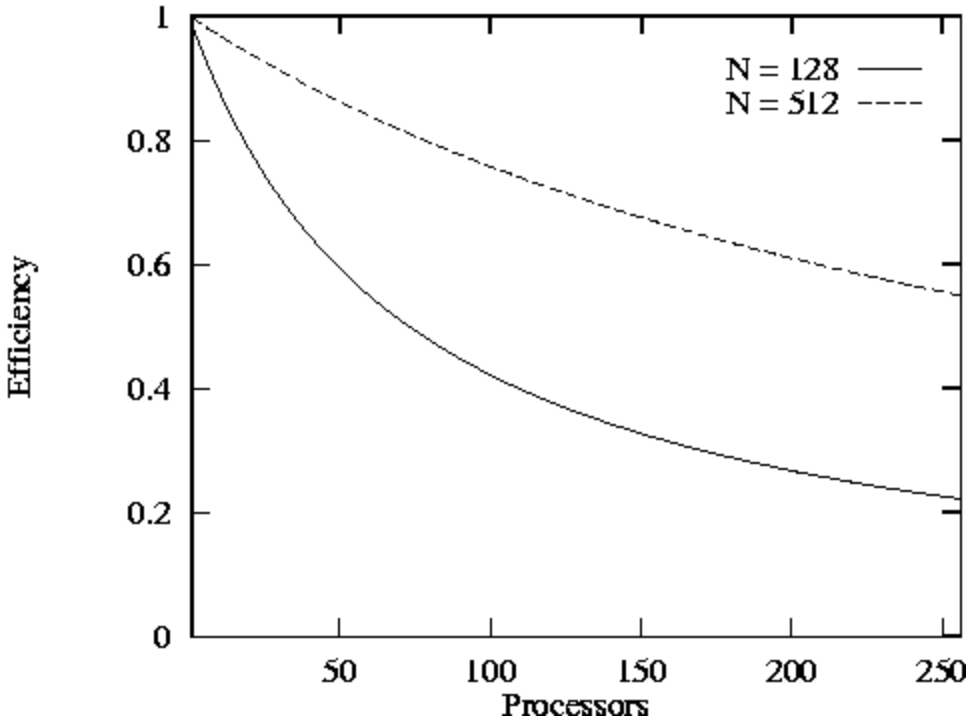
$E$ decresce monotonicamente con $P$ mentre $T$ può crescere se il modello include un termine proporzionale a una potenza positiva di $P$. *In tali casi non è conveniente usare più processori di un numero prefissato per una determinata dimensione del problema e scelta di macchina. L'algoritmo di load-balancing potrebbe utilizzare una opportuna look-up table per definire un mapping ottimale in run-time, a tale scopo.*

**Esempio: scalabilità delle differenze finite**

Nei grafici sono rappresentate $E$ e $T$ come funzione di $P$ considerando $t_c = 2\mu sec$




1e+07
1e+06
100000
10000
1000
100
Total Time
1
10
100
Processors
N = 128
(Ideal)
N = 512
(Ideal)


1
0.8
0.6
0.4
0.2
0
Efficiency
50
100
150
200
250
Processors
N = 128
N = 512

## Scalabilità con dimensione del problema scalabile

Un differente approccio è quello di stimare anche la scalabilità del problema oltre che rispetto al numero di processori impiegati, anche rispetto alla dimensione dell'input $N$. Si definisce in tal caso la funzione di *isoefficienza* ovvero la funzione di $N$ che mantiene $E$ costante al variare di $P$.

Un algoritmo con isoefficienza che è un $O(P)$ è altamente scalabile, dal momento che il calcolo cresce solo linearmente rispetto a $P$ per mantenere costante l'efficienza. Al contrario se l'algoritmo ha una isoefficienza quadratica o esponenziale rispetto a $P$ il programma sarà pessimamente scalabile.

L'analisi di scalabilità rispetto alle dimensioni del problema non ha sempre senso. Vincoli di real-time o dimensioni del problema fissate a priori possono rendere utile o inutile tale tipo di analisi.

## Esempio. Isoefficienza di algoritmi alle differenze finite

Si può usare l'analisi di isoefficienza per esaminare la scalabilità di due algoritmi d.f. La relazione seguente

$$t_c N^2 Z \approx E(t_c N^2 + t_s 2P + t_w 4NZP)$$



deve essere soddisfatta per $P$ crescente e $E$ costante. La funzione $N = P$ soddisfa il vincolo e porta alla relazione seguente, valida per $P$ grande quando il termine $t_s$ è trascurabile:

$$t_c Z \approx E \left( t_c Z + t_s \frac{2}{P} + t_w 4 Z + \right)$$

Dal momento che la griglia è quadrata, scalare $N$ con $P$ causa una aumento di $N^2$ del numero di punti (e del costo computazionale). Diciamo quindi che la funzione di isoefficienza per questo problema è $O(P^2)$ intendendo che l'ammontare del calcolo deve crescere quadraticamente rispetto al numero di processori per mantenere costante l'efficienza.

Altro esempio: si consideri una decomposizione bidimensionale del problema, con ogni task che gestisce $(N/\sqrt{P}) \times (N/\sqrt{P})$ punti e scambia $2(N/\sqrt{P})Z$ punti con ciascun vicino ad ogni step.

$$E = \frac{t_c N^2 Z}{t_c N^2 Z + t_s 4P + t_w 8 N Z \sqrt{P}}$$

La relazione da soddisfare per una efficienza costante, come



funzione di $P$ quando sostituita a $N$, risulta essere

$$t_c N^2 Z \approx E(t_c N^2 Z + t_s 4P + t_w 8NZ\sqrt{P})$$

La funzione $N = \sqrt{P}$ soddisfa la relazione e per ogni valore di $P$:

$$t_c Z \approx E(t_c Z + t_s 4 + t_w 8Z)$$

Poichè il calcolo in totale è di nuovo proporzionale a $N^2$, la funzione di isoefficienza è un $O(P)$. Questa analisi mostra che una decomposizione $2D$ è più scalabile di una decomposizione $1D$.



# Disegno modulare

L'idea base del disegno modulare è organizzare un sistema complesso come un insieme di componenti distinte (moduli) che interagiscono fra loro e possono essere sviluppate separatamente per poi essere unite.

- Fissare interfacce semplici per aumentare la riusabilità del software
- Assicurare l'information hiding
- Usare tool appropriati (scoping, funzioni, oggetti, allocazione dinamica)
- Checklist:
  1. Il disegno identifica moduli ben definiti?
  2. Ogni modulo ha un scopo ben definito?
  3. Le interfacce dei moduli sono sufficientemente chiare da non richiedere l'analisi della implementazione del modulo per comprenderle? I moduli nascondono effettivamente i dettagli implementativi?
  4. I moduli non replicano funzionalità?
  5. Sono stati isolati gli aspetti del disegno che sono hardware/software spcific?

Nella programmazione parallela un modolo deve incapsulare non solo dati e metodi, ma anche i task creati dai moduli, il



modo in cui le strutture dati sono partizionate e mappate ai processori e le strutture di comunicazione interna dei moduli. L'aspetto più essenziale è quello della distribuzione dei dati.

Si consideri inoltre che la composizione dei moduli è parallela e concorrente, non sequenziale come nella programmazione modulare o OO classica. *Non tutti i tool di programmazione parallela consentono qualsiasi tipo di composizione: i linguaggi data-parallel tendono a supportare solo la composizione sequenziale; le librerie di message-passing supportano la sequenziale e la parallela, ma non la concorrente; altri linguaggi supportano le tre forme di composizione*

Lo sviluppo dei moduli dovrebbe essere *data distribution neutral* Tale neutralità può essere ottenuta per esempio specificando in run-time la distribuzione dei dati e implementando in modo parametrico i moduli.

**Composizione sequenziale** In programmazione parallela si applica esclusivamente a programmi SPMD

```
while (!done) {
    finite_difference(localgrid, localmax);
    global_maximum(localmax, globalmax);
    if (globalmax < threshold) done = TRUE;
}
```



In un programma parallelo le due funzioni incapsulano la comunicazione inter-processore e la nascondono al resto del programma. Un vantaggio fondamentale della c.s. è la leggibiltà del programma e il fatto che non occorre strutturare le interfacce in modo da comprendere la comunicazione. Con la programmazione OO in realtà tale vantaggio è trascurabile.

*Esempio:* `ScaLAPACK`*, libreria di algebra lineare.*

**Composizione parallela** È una generalizzazione di SPMD in cui parti differenti di un computer eseguono programmi differenti. Una composizione parallela specifica quali componenti devono essere eseguiti e dove, come tali componenti scambiano i dati.

In linea di principio un programma espresso come composizione parallela può essere convertito in una composizione sequenziale che sincronizza appropriatamente le varie componenti.

**Composizione concorrente** È il tipo di composizione più generale, in quanto specifica le parti di programma che devono essere eseguite in concorrenza, le relazioni producer/consumer tra le componentie il mapping delle componenti sui processori.

